MODULE 26 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

Tracing with X-Ray and Application Signals

Segments, sampling, annotations - and the question that logs and metrics genuinely cannot answer.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 26 of 34

admin@sthithapragna.com

Logs say **what a service said**. Metrics say **how much and how often**. Only a trace says **where the time went**.

Traces, segments and subsegments

- A **trace** is the complete path of one request through your system, with one trace ID.
- A **segment** is one service's contribution to that trace, with timing and status.
- A **subsegment** is finer detail inside a segment: a database call, an HTTP request, a code block.
- Subsegments are what let you see that the time went to DynamoDB rather than your code.
- Every segment carries the same trace ID, which is how the pieces are reassembled.

> ON THE EXAM
>
> The subsegment level is where the answer usually is. A segment tells you a service was slow; a subsegment tells you which call inside it was slow.

How the trace ID propagates

- The **X-Amzn-Trace-Id** header carries the trace ID between services.
- The first service to see a request creates the ID. Everything downstream reuses it.
- The instrumented SDK adds the header to outbound calls automatically.
- A service that does not forward the header **breaks the trace** into disconnected pieces.
- **W3C Trace Context** is now supported alongside it, which matters for OpenTelemetry.

> ON THE EXAM
>
> A trace that stops halfway through the call chain is almost always a service that did not propagate the header, often because it was not instrumented at all.

The service map

- A generated diagram of your services, built from the segments X-Ray received.
- Each node is a service. Each edge is a call, labelled with latency and error rate.
- Nodes turn **yellow for 4xx** and **red for 5xx**, so failures are visible immediately.
- Clicking a node filters to the traces behind it.
- Nothing appears on the map unless it is instrumented, so gaps mean missing instrumentation.

> ON THE EXAM
>
> During an incident the service map is the fastest starting point: it shows which node is red before you have read a single log line.

Sampling

- Tracing everything would be expensive and would add overhead. X-Ray samples instead.
- The default rule: the **first request each second**, plus **5% of additional requests**.
- That one per second is the **reservoir**, guaranteeing at least one trace while traffic flows.
- At 1,000 requests per second that yields roughly 51 traces, about 5%.
- **So a specific failing request may simply not have been traced.**

> ON THE EXAM
>
> That last point causes real frustration. A customer reports a failure, you search for their trace, and it was never sampled - which is exactly what Transaction Search addresses.

Reading a trace

- The trace view is a **waterfall**: each segment and subsegment as a bar on a timeline.
- A long bar with short children means the time went to your own code.
- A long child bar means the time went to whatever that subsegment called.
- A **gap** between bars is time nothing accounted for, often queueing or a cold start.
- Status and response codes appear per segment, so a failure is located immediately.

> ON THE EXAM
>
> The gaps are the underrated part. Time that appears in the parent but in no child is unaccounted for, and that is usually where the surprise is.

Custom sampling rules

- Rules match on service name, host, HTTP method and **URL path**.
- Each sets a **reservoir size** and a **fixed rate**, and has a **priority**.
- Lower priority numbers win. The default rule sits at priority 10,000, so anything overrides it.
- Trace 100% of a payment endpoint, and 1% of health checks, from two rules.
- Rules are configured centrally and **fetched by the SDK**, so no redeployment is needed.

> ON THE EXAM
>
> The central configuration is the useful part: raising sampling on a problematic endpoint during an incident takes effect without shipping any code.

Annotations or metadata?

ANNOTATIONS

Indexed and searchable

- Key-value pairs, indexed by X-Ray
- Filterable in the console and API
- Strings, numbers and booleans only
- Up to 50 per trace
- Use for anything you will search on

METADATA

Stored, not indexed

- Arbitrary JSON of any shape
- Cannot be filtered on
- No practical count limit
- Read manually when viewing a trace
- Use for larger debug payloads

THE TIEBREAKER

The whole distinction is searchability. An order ID in metadata cannot be found; the same value as an annotation can be filtered on directly.

Tracing Lambda

- Enable **active tracing** on the function. No agent and no daemon required.
- The execution role needs the **AWSXRayDaemonWriteAccess** managed policy.
- That alone gives you the invocation segment, showing initialisation and invocation time.
- Adding the SDK or ADOT gives **subsegments** for downstream AWS and HTTP calls.
- Active tracing without instrumentation shows the function as a single opaque block.

> ON THE EXAM
>
> The two levels matter. Active tracing tells you the function took 900 ms; instrumentation tells you 850 ms of that was one DynamoDB query.

Tracing ECS and EC2

- On EC2 and ECS, something must **collect and forward** segments to X-Ray.
- The **X-Ray daemon** listens on **UDP port 2000** and batches segments to the service.
- On ECS it runs as a **sidecar container** in the same task definition.
- On EC2 it runs as a process on the instance, usually installed via user data.
- The **task role** or instance profile needs permission to write segments.

> ON THE EXAM
>
> This is the usual reason ECS services are missing from a service map: the application is instrumented, and nothing is listening on port 2000 to forward the segments.

Currency note: SDKs and daemon frozen

- **The examinable answer:** instrument with the X-Ray SDK, and run the daemon on ECS and EC2.
- On **25 February 2026** the X-Ray SDKs and the daemon entered **maintenance mode**.
- Security fixes only, no new features, and **end of support in February 2027**.
- **The X-Ray service itself is fully supported** and still being enhanced.
- The recommended path is **AWS Distro for OpenTelemetry**, or native OpenTelemetry.

> READ THIS TWICE
>
> Read the scope carefully. Only the instrumentation SDKs and the daemon are frozen. X-Ray, the service, is where Transaction Search and Application Signals are being built.

ADOT and OpenTelemetry

- **OpenTelemetry** is a vendor-neutral standard for traces, metrics and logs.
- **ADOT** is AWS's supported distribution of it, with an X-Ray exporter.
- Available as a **Lambda layer**, or as a collector sidecar on ECS and EC2.
- The same instrumentation can send data to X-Ray and to a third-party backend.
- **Answer X-Ray SDK on the exam.** ADOT postdates exam guide v2.1.

> ON THE EXAM
>
> The practical argument for OpenTelemetry is portability: instrumentation is no longer tied to one vendor's backend, which is why AWS itself moved to it.

What gets traced automatically

- **API Gateway** can enable tracing per stage, becoming the first segment.
- **Lambda** with active tracing, as covered. **Step Functions** per state machine.
- **SNS and SQS** propagate the trace header, so a trace survives an asynchronous hop.
- The instrumented SDK traces AWS API calls as subsegments automatically.
- Anything not instrumented appears as a gap, not as an error.

> ON THE EXAM
>
> The SQS propagation is worth knowing: without it, an asynchronous architecture would produce a separate disconnected trace for every queue hop.

Finding the trace you want

- Filter on **service()**, **http.status**, **responsetime**, **error**, **fault** and **throttle**.
- Filter on annotations directly: **annotation.orderId = "12345"**.
- **Error** is a 4xx, **fault** is a 5xx, and **throttle** is a 429. They are distinct.
- **Groups** save a filter expression and produce their own CloudWatch metrics.
- So you can alarm on the fault rate of one named group of traces.

> ON THE EXAM
>
> The error, fault and throttle distinction appears throughout X-Ray, including on the service map. Fault means your service failed; error means the caller did.

X-Ray Insights

- Detects anomalies in fault rates automatically, without a threshold being set.
- It builds a **baseline** of expected error rates and flags deviations from it.
- An insight identifies the affected service, the root cause node and the impact.
- Insights can notify through **EventBridge**, so they can page or open a ticket.
- Enabled per group, and it needs traffic to learn from.

> ON THE EXAM
>
> The value is identifying the root cause node rather than the symptomatic one. A slow database surfaces as errors in five services; Insights points at the database.

Limits worth knowing

- Traces are retained for **30 days**. There is no configurable retention.
- Maximum trace size is **500 KB**. A segment document is capped at **64 KB**.
- Up to **50 annotations** per trace.
- Encryption at rest uses a default key, or a KMS key you choose.
- Traces can be shared **across accounts** for a central observability account.

> ON THE EXAM
>
> The 30-day retention is the one that shapes practice: X-Ray is for investigating now, not for historical analysis. Long-term trace data belongs in logs.

X-Ray or CloudTrail?

AWS X-RAY

The data plane

- Traces application requests
- Where did this request spend time?
- Segments across your services
- Sampled, retained 30 days
- For performance and failure analysis

AWS CLOUDTRAIL

The control plane

- Logs AWS API calls
- Who deleted this bucket?
- One event per API call
- Complete, not sampled
- For audit and security investigation

THE TIEBREAKER

A named exam trap. They sound similar and answer opposite questions: X-Ray follows your traffic, CloudTrail records changes to your account.

Connecting traces to logs

- Lambda writes the **trace ID into the log line** when active tracing is on.
- So a slow trace links directly to the log events from that exact invocation.
- For your own services, log the trace ID yourself on every line.
- The X-Ray console links to CloudWatch Logs for the segment you are viewing.
- That closes the loop: alarm to metric, metric to trace, trace to log line.

> ON THE EXAM
>
> This is the payoff for structured logging from module 25. With a trace ID field, one Logs Insights filter retrieves every log line for a single request.

CloudWatch Application Signals

- Auto-instruments applications and produces a curated **application performance** view.
- Standard **golden metrics** per service and operation: latency, errors, faults, request volume.
- It discovers services automatically, including ones you have not instrumented.
- Built on X-Ray traces, so it is a layer above rather than a replacement.
- It defaults to **5% trace sampling**, adjustable up or down.

> ON THE EXAM
>
> The distinction from raw X-Ray is opinionated defaults. X-Ray gives you traces and expects you to know what to ask; Application Signals gives you the standard questions already answered.

X-Ray or Application Signals?

AWS X-RAY

Traces, and your questions

- Individual traces and the service map
- You choose what to instrument
- Filter expressions you write
- Annotations you decide on
- For investigating a specific problem

APPLICATION SIGNALS

Curated application view

- Golden metrics per operation
- Auto-instrumentation and discovery
- SLOs and error budgets
- Standard dashboards out of the box
- For knowing whether you are healthy

THE TIEBREAKER

They are layers, not alternatives. Application Signals reads X-Ray data, so enabling it does not replace anything you already have.

SLOs and error budgets

- An **SLO** is a target for a service level indicator over a window, such as p99 under 300 ms.
- The gap between target and 100% is the **error budget** you may spend.
- **Burn rate** is how fast you are consuming it. A rate of 1.0 exactly exhausts it over the window.
- Fast burn pages somebody. Slow burn opens a ticket. Two alarms, two urgencies.
- SLOs attach to a service or an operation, and track attainment automatically.

> ON THE EXAM
>
> Burn rate alarms fix the two failures of threshold alarms: paging for a brief spike that costs nothing, and staying silent through a slow degradation that costs everything.

Transaction Search

- Stores **100% of spans** as structured logs in a log group named **aws/spans**.
- Written in OpenTelemetry semantic convention format, queryable with Logs Insights.
- A configurable percentage, **1% by default**, is still indexed as X-Ray trace summaries.
- So a specific customer's failing request can be found, even though it was not sampled.
- Because spans land in Logs, metric filters and data masking work on trace data.

> ON THE EXAM
>
> This is the direct fix for 'the trace I need was never sampled'. Every span is stored; only the summary indexing is sampled.

Which observability tool?

THE SCENARIO SAYS	THE INTENDED ANSWER
Where did the latency go across services?	AWS X-Ray, and the service map
Which service is failing right now?	X-Ray service map, or Application Signals
Find the trace for order 12345	An X-Ray annotation, filtered on
Who changed this AWS resource?	AWS CloudTrail
Alert when errors exceed a count	A CloudWatch metric filter and alarm
Is the endpoint up, as a user sees it?	CloudWatch Synthetics canary
Are we meeting our latency target?	Application Signals SLO

ON THE EXAM

Worth screenshotting. Most Domain 4 selection questions reduce to one row, and the X-Ray against CloudTrail row is the one most often got wrong.

Ask first: does the scenario need one request followed, or everything counted?

> A checkout request passes through API Gateway, a Lambda function, an SQS queue, a second Lambda function and DynamoDB. Users report that checkout sometimes takes eight seconds. Every service reports healthy metrics and no errors appear in any log.
>
> The team needs to know which component is responsible before they can fix it.

What should they use?

A A CloudWatch dashboard combining the Duration metric of both Lambda functions

B A Logs Insights query across all the log groups, sorted by timestamp

C CloudTrail, to see which API calls were made during the slow requests

D AWS X-Ray with active tracing enabled across the chain, and the service map

◆ PAUSE AND DECIDE

What should they use?

A Dashboard of Duration metrics

Averages across all requests. The slow ones are a minority, so the metric barely moves, which is why everything already looks healthy.

B Logs Insights across log groups

Log lines from different services cannot be tied to one request without a shared trace or correlation ID. You would be matching on timestamps and guessing.

C CloudTrail

Records AWS API calls for audit. It has no visibility into how long an application request spent in each component.

D X-Ray and the service map

A trace follows one request through every hop with timing per segment, which is exactly the question. The service map shows which edge is slow.

THE SIGNAL

'Every service reports healthy metrics' is the clue: aggregate metrics cannot see a minority of slow requests, which is precisely the gap tracing fills.

> A support team needs to investigate individual failed orders. Given an order ID, they must retrieve the X-Ray trace for that specific order. The application currently attaches the order ID and the full order payload to each segment using put_metadata.
>
> Searching for the order ID in the X-Ray console returns nothing.

What change is needed?

A Increase the sampling rate to 100% so all orders are traced

B Record the order ID as an annotation instead, keeping the payload as metadata

C Move the order ID into the segment name so it appears on the service map

D Export traces to S3 and query them with Athena

◆ PAUSE AND DECIDE

What change is needed?

A 100% sampling

Would trace every order, and the traces still could not be searched by order ID, because metadata is not indexed. It also multiplies cost.

B Order ID as an annotation

Annotations are indexed and filterable, so annotation.orderId = the value retrieves the trace. The bulky payload correctly stays as metadata.

C Order ID in the segment name

Segment names identify services, not requests. A unique name per order would create thousands of nodes on the service map.

D Export and query with Athena

Builds a search system to work around an indexing feature that already exists and needs one line changed.

THE SIGNAL

Note that B keeps the payload as metadata. The fix is not moving everything to annotations, but putting the searchable field in the searchable place.

> An application runs as an ECS service on Fargate. The code is instrumented with the X-Ray SDK and the same code traces correctly when run as a Lambda function. On ECS, no segments appear in X-Ray at all and the service is missing from the service map.
>
> The task role has the AWSXRayDaemonWriteAccess policy attached.

What is missing?

A Active tracing must be enabled on the ECS service, as it is on Lambda

B The task execution role needs X-Ray permissions rather than the task role

C The X-Ray daemon is not running as a sidecar container in the task definition

D Fargate does not support X-Ray tracing

◆ PAUSE AND DECIDE

What is missing?

A Enable active tracing

Active tracing is a Lambda function setting. ECS services have no equivalent toggle, because collection is the daemon's job.

B Task execution role

Module 14's distinction: the execution role starts the container, the task role is what application code uses. Writing segments is application code.

C No daemon sidecar

The SDK sends segments to UDP port 2000 and something must listen there. On ECS that is a daemon sidecar in the task definition, which is absent.

D Fargate unsupported

Fargate supports X-Ray fully, using a sidecar container exactly as EC2-launch-type tasks do.

THE SIGNAL

'The same code works on Lambda' is the clue: the instrumentation is fine, so the difference is collection. Lambda collects for you; ECS does not.

A high-traffic API is traced with default sampling. When a customer reports a failure with a specific request ID, the trace almost never exists, because that request was not sampled. Raising sampling to 100% was rejected as too expensive.

The team needs to investigate individual reported failures, and still wants the normal service map and trace summaries.

What should they enable?

A CloudWatch Transaction Search, which stores 100% of spans in the aws/spans log group

B A custom sampling rule with a larger reservoir on the failing endpoint

C X-Ray Insights, to detect the anomalies automatically

D A CloudWatch metric filter counting failures by request ID

◆ PAUSE AND DECIDE

What should they enable?

A Transaction Search

Stores every span as structured logs while indexing only a small percentage as summaries. Individual requests become findable, and the normal experience is unchanged.

B Larger reservoir

Raises the proportion sampled without guaranteeing any particular request. The reported one may still be missing, which is the actual complaint.

C X-Ray Insights

Finds anomalies across traffic. It cannot retrieve a trace that was never recorded.

D Metric filter by request ID

Request ID is unbounded cardinality, so this would create a metric per request. Module 25 ruled that out explicitly.

THE SIGNAL

The requirement is a specific request, not a representative sample. Only storing everything satisfies that, and Transaction Search separates storing from indexing.

What to carry forward

- **Only a trace answers where the time went.** Metrics average it away.
- **Annotations are indexed, metadata is not.** Anything you will search on is an annotation.
- **Lambda needs only active tracing. ECS and EC2 need the daemon**, as a sidecar or a process.
- Default sampling is 1 per second plus 5%, so a given request may not be traced.
- X-Ray is the data plane, CloudTrail is the control plane. A named exam trap.

> ON THE EXAM
>
> Next: troubleshooting errors, throttling and retries - which uses everything from these two modules on the failures the exam actually asks about.